

Memoria Práctica 2

Aprendizaje Automático

Jorge Alcalde Vesteiro
Andy Barcia Rodríguez
Pedro Pablo López Mena

Índice

1) Introducción	3
2) Cuestiones Previas	3
3) Implementación	3
3.a) Nodo	3
3.b) <i>Threshold</i>	5
3.c) Funciones de información	6
3.d) Árbol	7
4) Pruebas	11
4.a) Base PDF	11
4.b) Comparación con Scipy	13
4.c) Tuning de hiperparámetros y <i>threshold</i>	15
4.d) Gráficos de Pareto	15
4.e) Comparación con un MLP	17
5) Conclusión	21

1) Introducción

El contenido de esta práctica consiste en construir y evaluar un modelo de caja blanca. En este caso un árbol de decisión.

La idea de la práctica es observar como los modelos como los árboles son transparentes de tal manera que las persona que los construye sabe cual es la razón de las decisiones del modelo. Frente a esto los modelos de caja negra destacan por ser difíciles de interpretar.

A mayores, en esta práctica se intenta introducir un debate entre la interpretabilidad y la precisión, buscando encontrar un balance entre ambas.

2) Cuestiones Previas

Vamos a trabajar con dos *datasets*

- Lenses: es un *dataset* que nos indica si una persona requiere o no el uso de lentillas en base a varios parámetros. Es un *dataset* con tres clases y 24 ejemplos. A mayores encontramos una variación en la que una de las columnas es numérica para evaluar el *dataset* con *thresholds*.
- Iris: es un *dataset* que clasifica las flores en tres clases en base a diversos parámetros numéricos. Cuenta con 150 ejemplos para poder evaluar.

Podemos observar que realmente ambos *datasets* son de un tamaño pequeño. De esta manera será complicado evaluar el modelo de manera concisa por la falta de ejemplos. De la misma manera, un *dataset* pequeño favorece la interpretabilidad de manera intrínseca y dificulta intentar crear un árbol complejo.

Respecto a los modelos:

- Vamos a trabajar con dos tipos de árboles, el que está creado por nosotros mismos y su variante de sklearn. Ya se explicará en más detalle en apartados posteriores, pero la manera de tratar los *thresholds* en nuestro árbol se ha planteado de una manera diferente a la de sklearn, sin embargo a la hora de realizar la comparación se crearán *thresholds* equivalentes para verificar que funcionan igual.
- Vamos a trabajar con un MLP. El único detalle a mencionar de este modelo es que realmente los MLPs son muy sensibles a los datos por lo que es posible que para poder evaluar este modelo haga falta un mínimo de preprocesado, como una normalización.

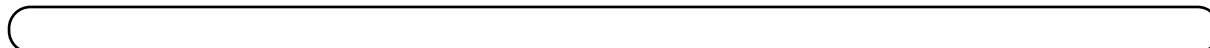
3) Implementación

3.a) Nodo

La base de la implementación es una clase *Nodo* que representa:

- Los nodos de decisión internos del árbol. Éstos guardan el atributo sobre el que se realiza la división del espacio de búsqueda, y los umbrales que se utilizan para la división. Dichos umbrales pueden ser un número, o pueden ser una lista de clases concretas.
- Los nodos hoja. Éstos representan la predicción final del árbol de decisión, por lo que guardan todas las predicciones junto a sus probabilidades.

Adicionalmente, se implementan algunas cuantas funciones para determinar la profundidad máxima, mínima, y media del árbol, además de otra función para contar el número de nodos hoja total.



```

class Node:
    def __init__(
        self,
        value : Optional[List] = None,
        probs : Optional[List[float]] = None,
        children : List['Node'] = [],
        feature : Optional[int] = None,
        threshold : Optional[Union[float, List]] = None,
    ):
        self.value = value
        self.probs = probs
        self.children = children
        self.feature = feature
        self.threshold = threshold

        if value is not None and len(value) > 0:
            self._is_leaf = True
        else:
            self._is_leaf = False

    def is_leaf(self):
        return self._is_leaf

    def max_depth(self):
        return 0 if self._is_leaf else max([1+c.max_depth() for c in self.children])

    def min_depth(self):
        return 0 if self._is_leaf else min([1+c.min_depth() for c in self.children])

    def avg_depth(self):
        return 0 if self._is_leaf else avg([1+c.avg_depth() for c in self.children])

    def num_leafs(self):
        return 1 if self._is_leaf else sum([c.num_leafs() for c in self.children])

```

Además de estas funciones de ayuda, la clase cuenta con una función muy importante de predicción. Ésta es la que recorre el árbol de decisión en base a los valores de los umbrales. Se puede elegir que se devuelvan las probabilidades de las predicciones, o que se devuelva la predicción más probable.

```

class Node:
    [...]

    def _predict(self, x : List[Any], verbose=False, get_probs=True):
        if self.is_leaf():
            # Si es un nodo hoja, simplemente se devuelve las probabilidades.
            if get_probs:
                return (self.value, self.probs)
            else:
                return self.value[np.argmax(self.probs)]
        elif isinstance(self.threshold, float):
            if verbose:
                print(x, self.feature, self.threshold)
            # Si es un nodo de decisión basado en un umbral, coger uno de los
            # dos hijos en base a si el valor es mayor o inferior al umbral.

```

```

idx_child = int(x[self.feature] > self.threshold)
return self.children[idx_child].predict(x, verbose, get_probs)
else:
    if verbose:
        print(x, self.feature, self.threshold)
    # Si es un nodo de decisión, y el umbral es una lista de valores
    # discretos, buscar en el hijo que corresponda a la clase.
    idx_child = self.threshold.index(x[self.feature])
    return self.children[idx_child].predict(x, verbose, get_probs)

```

3.b) Threshold

Una segunda clase de importancia para la implementación sería la clase *threshold*. Ésta se utiliza para separar la lógica de construcción del árbol de la lógica de propuestas de umbrales para la generación de splits del espacio de búsqueda.

La clase ofrece las siguientes posibilidades:

1. La opción «avg» permite probar como *thresholds* de los atributos los valores que estén en la media, una desviación típica por encima de la media, y una desviación típica por debajo de la media.
2. La opción de «custom», que se pedía implementar de forma obligatoria. En este modo el usuario puede especificar los *thresholds* que quiere que se prueben para cada uno de los atributos. En caso de que no se propongan *thresholds* para todos los atributos, se rellenarán los faltantes con la estrategia «avg».
3. La opción «best», que es la que da mejores resultados al probar mayor cantidad de umbrales. En esta opción el árbol probará con todos los *thresholds* basados en los términos medios de los valores de un feature obteniendo así los mejores resultados. De esta manera, con «best» si tuviéramos un array de valores (1, 2, 3) los *thresholds* que se usarían serían (1.5, 2.5). Esta función, o una variación de la misma es la que implementa el árbol de scipy por defecto.

```

class Theshold:
    def __init__(
        self,
        type : Literal["best", "custom", "avg"] = 'best',
        th_array : List[List] = None
    ) -> None:
        self.type = type
        self.th_array = th_array

    def _fit_array(self, data : np.array):
        if self.type == "best":
            # Para cada columna añadir todos los valores posibles.
            self.th_array = []
            for i in range(data.shape[1]-1):
                self.th_array.append(np.unique(data[:,i]))
            # Para cada columna añadir todos los valores posibles.
            self.th_array = []
            for i in range(data.shape[1]-1):
                # Valores únicos ordenados
                values = np.unique(data[:,i])
                # Establecer los umbrales como la media entre valores contiguos
                self.th_array.append((values[:-1] + values[1:]) / 2)

```

```

elif self.type == "custom":
    # Comprobar que se han especificado una cantidad suficiente
    # de umbrales para los atributos continuos. En el caso de
    # que no se hayan indicado atributos para todas las columnas,
    # rellenar las que faltan con la estrategia "avg".
    size_flag = 0
    for i in range(data.shape[1]-1):
        if type(data[:,i][0]) == str:
            continue
        elif i < len(self.th_array):
            th_column_tam = len(self.th_array[i])
            if th_column_tam < 3:
                warnings.warn(f"threshold {i} column tiene pocos valores")
        else:
            size_flag = 1
            media, stdev = np.mean(data[:,i]), statistics.stdev(data[:,i])
            self.th_array.append([media - stdev, media, media + stdev])
    if size_flag:
        warnings.warn(f"custom threshold columnas faltantes, autorelleno con
avg type")
elif self.type == "avg":
    # Para cada columna que no sea un atributo discreto,
    # añadir como umbral la media, y la desviación estándar
    # por encima y por debajo de la media.
    self.th_array = []
    for i in range(data.shape[1]-1):
        if type(data[:,i][0]) == str:
            self.th_array.append([])
        else:
            media, stdev = np.mean(data[:,i]), statistics.stdev(data[:,i])
            self.th_array.append([media - stdev, media, media + stdev])

```

3.c) Funciones de información

Para realizar las decisiones de cuando realizar o no un split se emplea el método de la ganancia de información. Para calcular la cantidad información o «desorden» que hay en un conjunto de datos pueden utilizar dos métodos:

- La entropía. Esta es 0 cuando el conjunto de datos es perfectamente uniforme (todos los datos tienen la misma etiqueta), y es mayor cuanto mayor sea la cantidad de etiquetas distintas.
- La impuridad de Gini. Este calcula la probabilidad de que una etiqueta elegida aleatoriamente sea clasificada incorrectamente al elegir otra etiqueta aleatoriamente. Si todas las etiquetas son iguales, la probabilidad es 0. Si todas las etiquetas son distintas, la probabilidad es 1.

```

def entropy(y : np.array) -> float:
    labels, label_counts = np.unique(y, return_counts=True)

    entropy = 0.0
    if len(labels) <= 1:
        return entropy

    num_samples = len(y)
    probs = label_counts / num_samples

```

```

    for prob in probs:
        entropy += -prob * np.log2(prob)

    return entropy

def gini(y : np.array) -> float:
    num_samples = len(y)
    labels, label_counts = np.unique(y, return_counts=True)

    gini = 0.0
    if len(labels) <= 1:
        return gini

    probs = label_counts / num_samples
    for prob in probs:
        gini += prob*prob

    return 1 - gini

```

Information Gain es la función que calcula la ganancia real de información, en este caso comparando (restando) la entropía de los hijos de la del padre. Es decir calcula la diferencia de entropía que es igual a la ganancia de información

```

def information_gain(
    x : np.array,
    x_division : List[np.array],
    type : Literal["entropy", "gini"] = "entropy"
):
    """
    Calcula la ganancia de información de dividir un conjunto de etiquetas en
    varios subconjuntos, determinado como la información del conjunto completo,
    menos la información de cada uno de los subconjuntos.
    """

    ratios = np.array([len(division) / len(x) for division in x_division])
    if type == "entropy":
        return entropy(x) - sum(ratios * np.array([entropy(division) for division
in x_division]))
    if type == "gini":
        return gini(x) - sum(ratios * np.array([gini(division) for division in
x_division]))

```

3.d) Árbol

Por último, presentamos la clase que da vida al algoritmo, en este caso la clase «DecisionTree» implementada por nosotros.

Para definir el árbol usamos los siguientes criterios:

- `max_depth`: define la profundidad máxima que puede tener el árbol que generamos
- `min_samples_split`: define la cantidad mínima de elementos que tiene que tener un nodo para poder seguir dividiéndose
- `min_samples_leaf`: define la mínima cantidad de elementos que tiene que tener el split usado para generar la hoja

- `min_information_gain`: define el ratio mínimo de ganancia que se puede tener por iteración antes de que pare el algoritmo
- `criterion`: se usa para seleccionar la manera de aproximar el «information gain» explicado previamente

```
class DecisionTree:
    def __init__(
        self,
        min_info_gain:float,
        max_depth:Optional[int]=None,
        min_samples_split:int=2,
        min_samples_leaf:int=1,
        criterion:Literal["entropy","gini"] = "entropy"
    ):
        self.min_info_gain = min_info_gain
        self.min_samples_split = min_samples_split
        self.max_depth = max_depth
        self.min_samples_leaf = min_samples_leaf
        self.criterion = criterion

        assert self.min_samples_split > self.min_samples_leaf
```

Para el empleo del árbol podemos dividir las funciones en dos grandes grupos. Aquellas que sirven para construir el árbol y aquellas que sirven para recorrerlo.

Las funciones de construcción del árbol se basan en el pseudocódigo explicado a continuación:

1) La función principal es «fit», pero «fit» simplemente inicializa los *thresholds* y llama a «build_tree», por lo que vamos a considerar esta función como el backbone de la creación

En esta función lo que hacemos es de manera recursiva realizar los siguiente:

- Comprueba que no hayamos inclumplido ninguno de los criterios de parada. Es decir, comprueba que no nos pasemos de la profundidad máxima, que mantengamos los tamaños de los splits y comprueba la ganancia de esta iteración. En caso de que alguno de los criterios de parada se cumpla, generamos un nodo hoja con los valores que nos devuelva la función «get_leaf_value» que se explicará a continuación
- En caso de que no tengamos de parar porque nos se requiere según los criterios, lo que hacemos es usar a la función «best_split» para obtener los splits y la information gain que tendrá la siguiente capa del árbol
- En caso de avanzar se construye un array con los hijos a partir de los splits del punto anterior y una vez generados se crea un nodo decisión en base al threshold usado para esos splits

```
class DecisionTree:
    [...]
    def _build_tree(self, data:np.array, threshold_array:Theshold, depth=0) -> Node:
        """
        Genera recursivamente un nodo del árbol de decisión a partir de los
        datos de un nodo y los umbrales a probar para dividir el espacio de búsqueda.
        """

        # Comprobar si superamos la profundidad máxima. Si sí, forzamos un nodo
```

```

hoja.
    if self.max_depth and depth >= self.max_depth:
        value, probs = self._get_leaf_value(data)
        return Node(value=value, probs=probs)

    n_samples, n_features = np.shape(data)

    # Comprobar si tenemos datos suficientes como para dividir el espacio de
    # búsqueda. Si no, forzamos un nodo hoja.
    if n_samples <= self.min_samples_split:
        value, probs = self._get_leaf_value(data)
        return Node(value=value, probs=probs)

    # Buscar el umbral óptimo para dividir el espacio de búsqueda.
    data_splits, info_gain, th, best_index = self._best_split(data,
threshold_array)

    # Comprobar que la división del espacio de búsqueda nos hace ganar
    # suficiente información. Si no, forzamos un nodo hoja.
    if info_gain <= self.min_info_gain:
        value, probs = self._get_leaf_value(data)
        return Node(value=value, probs=probs)

    # Comprobar que cada una de las divisiones del espacio de búsqueda
    # tiene suficientes datos. Si no, no realizamos la división.
    for data_split in data_splits:
        n_samples, n_features = np.shape(data_split)
        if n_samples < self.min_samples_leaf:
            value, probs = self._get_leaf_value(data)
            return Node(value=value, probs=probs)

    # Si todo está bien, generar de forma recursiva los nodos de cada
    # una de las divisiones del espacio de búsqueda, y terminar.
    children = [
        self._build_tree(data_split, threshold_array, depth+1)
        for data_split in data_splits
    ]

    return Node(
        children=children,
        threshold=th,
        feature=best_index,
    )

[...]
```

```

def fit(self, data:Union[np.array,pd.DataFrame], threshold_array:Theshold):
    data_tn = data.to_numpy()
    threshold_array._fit_array(data_tn)
    self.root = self._build_tree(data_tn, threshold_array = threshold_array)

```

2) La función «best_split» es la que usamos para devolver al constructor del árbol los mejores splits que se han encontrado para la iteración actual. Para ello lo que hace es ir recorriendo cada uno de los «features» generando todos los posibles splits para ese «feature» y seleccionando su mejor *threshold*. Y luego para cada mejor *threshold* de cada feature nos quedamos con el mejor global usando un bucle for estándar

```

class DecisionTree:
[...]
```

`def _best_split(self, data:np.array, threshold_array:Theshold):`

```

    """
    Determina la forma de dividir un conjunto de datos que aumente
    más la cantidad de información ganada.

    Para ello se prueba a dividir cada atributo del conjunto de datos,
    y se prueban todos los umbrales especificados por threshold_array.
    """

    best_information_gain = -math.inf
    for i in range(len(data[0])-1):
        split, th = self._create_split(data, i, threshold_array)
        gain = information_gain(data[:, -1], [s[:, -1] for s in split],
type=self.criterion)
        if gain > best_information_gain:
            best_split, best_information_gain, best_th, best_index = split,
gain, th, i
    return best_split, best_information_gain, best_th, best_index

```

3) La función «create_split» realiza la tarea de generar todos los posibles splits para un «feature» concreto en base a los *thresholds* que hayamos definido. Para ello recorreremos con un bucle for todos esos splits calculando su information gain y guardamos el mejor.

De esta forma, de manera local escogemos cual es el mejor *threshold* para la feature actual en base al split padre que se nos ha pasado. En este caso cabe además destacar el hecho de que los *thresholds* se usan para variables numéricas, en caso de tener una variable definida como clases, se emplearán las clases como los delimitadores de los splits en cada caso.

```

class DecisionTree:
[...]
```

`def _create_split(`

```

    self,
    x : np.array,
    column_id,
    threshold_array : Theshold
) -> tuple[list, Union[float, list]]:
```

```

    """
    Dado los datos de un nodo y posibles umbrales, obtiene la mejor
    división posible de los datos (según la mayor cantidad de
    información ganada).

    Se devuelven por un lado la lista de los datos separados, y el umbral
    elegido para separar los datos.
    """

    x_column = x[:, column_id]
    if type(x_column[0]) == str:
        classes = np.unique(x_column)
        return [x[x_column == name] for name in classes], classes.tolist()
    else:
        best_gain, best_th, best_left, best_right = -math.inf, None, None, None
        for th in threshold_array.th_array[column_id]:

```

```

        x_left = x[x_column <= th]
        x_right = x[x_column > th]
        if len(x_left) < 1:
            warnings.warn(f"Threshold on column {column_id} was autobalanced
(all weight was on one Node)")
            x_left, x_right = x_right[0:1], x_right[1:]
        if len(x_right) < 1:
            warnings.warn(f"Threshold on column {column_id} was autobalanced
(all weight was on one Node)")
            x_right, x_left = x_left[-2:-1], x_left[:-1]
        gain = information_gain(x[:, -1], [x_left[:, -1], x_right[:, -1]])
        if gain > best_gain:
            best_gain, best_th, best_left, best_right = gain, th, x_left,
x_right
    return [best_left, best_right], best_th

```

4) Por último la función «get_leaf_value» lo que consigue es devolver el valor que habría que poner a los nodos hoja. En este caso definimos value como la combinación de clase + probabilidad que tengamos en el nodo. De esta manera, si en este nodo hoja solo hay elementos de la clase Y, value sería (Y, 100%) mientras que si hay 1 elemento de la clase X y dos de Y, value sería ([X,Y], [33%, 66%]). A la hora de recorrer el árbol, el elemento que se seleccionará como valor es el que tenga la probabilidad más alta.

```

class DecisionTree:
[... ]
    def _get_leaf_value(self, x:np.array) -> tuple[np.array,np.array]:
        """
        Dado los datos de un nodo, devuelve las predicciones
        posibles junto a sus sendas probabilidades.
        """

        y = x[:, -1]
        lables, label_counts = np.unique(y, return_counts=True)
        return lables, label_counts/len(y)

```

Por otro lado, como las funciones de predict se ayudan de las funciones de los nodos, estas acaban siendo funciones que llaman a métodos explicados anteriormente

```

class DecisionTree:
[... ]
    def predict(self, X, verbose=False, get_probs=True):
        return self.root.predict(X, verbose, get_probs)

    def predict_all(self, X, verbose=False, get_probs=True):
        return self.root.predict_all(X, verbose, get_probs)

```

4) Pruebas

A continuación vamos a realizar una serie de pruebas para comprobar que el arbol funciona

4.a) Base PDF

Lo primero que vamos a hacer es construir el arbol base de lentes que se nos propuso en el PDF

```
mi_tree = DecisionTree(
    min_info_gain=0,
    max_depth=None,
    min_samples_split=1,
    min_samples_leaf=0,
)
th = Theshold(type="custom", th_array=[[1.5], [0.5], [0.5], [0.5]])
mi_tree.fit(data, threshold_array=th)
```

Vemos que construye un árbol análogo

```
|> (root) Decision Node - Divided by threshold 0.5 on feature[:,3] (dataset column)
|--> Leaf node (<= 0.5)
|   |--> class: ['no lenses'] with probs [1.]
|--> (> 0.5) Decision Node - Divided by threshold 0.5 on feature[:,2] (dataset
column)
|-----> (<= 0.5) Decision Node - Divided by threshold 1.5 on feature[:,0] (dataset
column)
|-----> Leaf node (<= 1.5)
|         |--> class: ['soft'] with probs [1.]
|-----> (> 1.5) Decision Node - Divided by threshold 1.5 on feature[:,0] (dataset
column)
|-----> Leaf node (<= 1.5)
|         |--> class: ['no lenses'] with probs [1.]
|-----> Leaf node (> 1.5)
|         |--> class: ['soft'] with probs [1.]
|-----> (> 0.5) Decision Node - Divided by threshold 0.5 on feature[:,1] (dataset
column)
|-----> Leaf node (<= 0.5)
|         |--> class: ['hard'] with probs [1.]
|-----> (> 0.5) Decision Node - Divided by threshold 0.5 on feature[:,1] (dataset
column)
|-----> Leaf node (<= 0.5)
|         |--> class: ['hard'] with probs [1.]
|-----> Leaf node (> 0.5)
|         |--> class: ['no lenses'] with probs [1.]
```

A continuación realizamos la misma prueba pero con las edades de manera numérica

```
warnings.filterwarnings("ignore")
mi_tree_n = DecisionTree(
    min_info_gain=0,
    max_depth=None,
    min_samples_split=1,
    min_samples_leaf=0,
)
th = Theshold(type="custom", th_array=[[20,25,30,35,40,45], [0.5], [0.5], [0.5]])
mi_tree_n.fit(data_n, threshold_array=th)

print(f"Max depth: {mi_tree_n.root.max_depth()}")
print(f"Min depth: {mi_tree_n.root.min_depth()}")
print(f"Avf depth: {mi_tree_n.root.avg_depth()}")
print(f"Num leafs: {mi_tree_n.root.num_leafs()}")
```

De nuevo podemos observar que construye un árbol correcto, pero ahora considerando las edades como números y no clases

```
|> (root) Decision Node - Divided by threshold 0.5 on feature [:,3] (dataset column)
|--> Leaf node (<= 0.5)
|   |--> class: ['no lenses'] with probs [1.]
|--> (> 0.5) Decision Node - Divided by threshold 0.5 on feature [:,2] (dataset
column)
|----> (<= 0.5) Decision Node - Divided by threshold 40 on feature [:,0] (dataset
column)
|-----> Leaf node (<= 40)
|         |--> class: ['soft'] with probs [1.]
|-----> (> 40) Decision Node - Divided by threshold 20 on feature [:,0] (dataset
column)
|-----> Leaf node (<= 20)
|         |--> class: ['no lenses'] with probs [1.]
|-----> Leaf node (> 20)
|         |--> class: ['soft'] with probs [1.]
|----> (> 0.5) Decision Node - Divided by threshold 0.5 on feature [:,1] (dataset
column)
|-----> Leaf node (<= 0.5)
|         |--> class: ['hard'] with probs [1.]
|-----> (> 0.5) Decision Node - Divided by threshold 20 on feature [:,0] (dataset
column)
|-----> Leaf node (<= 20)
|         |--> class: ['hard'] with probs [1.]
|-----> Leaf node (> 20)
|         |--> class: ['no lenses'] with probs [1.]
```

4.b) Comparación con Scipy

Vamos a comprobar que funciona como scipy. Para ello lo primero que tenemos que hacer es «ajustar» el modelo para que sea equivalente. Para ello hacemos que el árbol de scipy use «entropy» para la información y que el árbol nuestro use «best» como estrategia (Mismos parámetros para el resto de valores). A mayores, usaremos el «dataset» Iris.csv para no repetirnos con los *datasets* de lentes del apartado anterior.

Vemos como se construyen árboles análogos (Para esta prueba se usa un Train_test_split básico):

```
dtc_a = DecisionTreeClassifier(criterion="entropy", max_depth=10)
dtc_b = DecisionTree(max_depth=10, min_info_gain=0, min_samples_split=2,
min_samples_leaf=1)
```

```
print(tree.export_text(dtc_a))

|--- feature_3 <= 0.75
|   |--- class: 0
|--- feature_3 > 0.75
|   |--- feature_3 <= 1.75
|   |   |--- feature_2 <= 4.95
|   |   |   |--- feature_3 <= 1.65
|   |   |   |   |--- class: 1
|   |   |   |   |--- feature_3 > 1.65
|   |   |   |   |   |--- class: 2
```

```

| | | |--- feature_2 > 4.95
| | | |--- feature_3 <= 1.55
| | | |--- class: 2
| | | |--- feature_3 > 1.55
| | | |--- feature_2 <= 5.45
| | | |--- class: 1
| | | |--- feature_2 > 5.45
| | | |--- class: 2
| | |--- feature_3 > 1.75
| | |--- class: 2

```

```

dtc_b.print_tree()

|> (root) Decision Node - Divided by threshold 2.5999999999999996 on feature[:,2]
(dataset column)
|--> Leaf node (<= 2.5999999999999996)
|   |--> class: [0.] with probs [1.]
|---> (> 2.5999999999999996) Decision Node - Divided by threshold 1.75 on feature
[:,3] (dataset column)
|-----> (<= 1.75) Decision Node - Divided by threshold 4.95 on feature[:,2] (dataset
column)
|-----> (<= 4.95) Decision Node - Divided by threshold 1.65 on feature[:,3] (dataset
column)
|-----> Leaf node (<= 1.65)
|         |--> class: [1.] with probs [1.]
|-----> Leaf node (> 1.65)
|         |--> class: [2.] with probs [1.]
|-----> (> 4.95) Decision Node - Divided by threshold 1.55 on feature[:,3] (dataset
column)
|-----> Leaf node (<= 1.55)
|         |--> class: [2.] with probs [1.]
|-----> (> 1.55) Decision Node - Divided by threshold 6.75 on feature[:,0]
(dataset column)
|-----> Leaf node (<= 6.75)
|         |--> class: [1.] with probs [1.]
|-----> Leaf node (> 6.75)
|         |--> class: [2.] with probs [1.]
|-----> Leaf node (> 1.75)
|         |--> class: [2.] with probs [1.]

```

De hecho, si comparamos los accuracies, con el test vemos que coinciden.

```

y_pred_a = dtc_a.predict(X_test)
y_pred_b = dtc_b.predict(X_test)

# Accuracy of sklearn decision tree
metrics.accuracy_score(y_true=Y_test, y_pred=y_pred_a)

0.9736842105263158

# Accuracy of our decision tree
metrics.accuracy_score(y_true=Y_test, y_pred=y_pred_b)

0.9736842105263158

```

```
# Comparasion of the predictions of both decision trees
metrics.accuracy_score(y_true=y_pred_a, y_pred=y_pred_b)

1.0
```

Las únicas diferecias que podría haber seguramente vinieran del modo en el que scipy hace el sampling de datos (que construye el array de *thresholds* en orden aleatorio y excluye algunos datos en base a criterios concretos)

4.c) Tuning de hiperparámetros y *threshold*

Para este apartado vamos a volver a los *datasets* de lentes (*lenses.csv* y *lenses2.csv*)

4.d) Gráficos de Pareto

En este apartado vamos a ajustar los hiperparámetros del árbol con *cross validation*, para después presentar gráficamente cómo varía la precisión frente al tamaño de los árboles finales generados. Se probarán todas las combinaciones validas de los siguientes parámetros:

```
hiperparametros = {
    "min_info_gain": (0,0.05,0.1),
    "max_depth": (2,3,4,5,6,7,8,9,10),
    "min_samples_split": (2,3,4,5,6,7,8,9,10),
    "min_samples_leaf": (1,2,3,4,5,6,7,8,9,10),
}
results = CV(X, Y, hiperparametros, splits=4, th=Threshold(type="custom", th_array=[
    [0.5, 1.5], # Age [0-2]
    [0.5], # Prescription [0-1]
    [0.5], # Astigmatic [0-1]
    [0.5], # Tear Rate [0-1]
]))
results = pd.DataFrame(results)
```

Los resultados obtenidos se muestran en la Figura 1. Se puede ver que, pese a que se realizan cientos de entrenamientos distintos, con distintos hiperparámetros y distintos subconjuntos del dataset, se generan poco más de 10 árboles únicos, con una precisión que ronda desde el 50% hasta el 100%.

Lo que queda más claro de los gráficos es que para conseguir una precisión razonable hace falta un árbol de, como mínimo, 3 hojas, y una profundidad media de 1.5 (es decir, alguna rama de al menos profundidad 2). De hecho, dicho árbol mínimo se corresponde con el siguiente:

```
> (root) Decision Node - Divided by threshold 0.5 on feature[:,3] ('TearRate')
|--> Leaf node (<= 0.5)
| |--> class: ['no lenses'] with probs [1.]
|--> (> 0.5) Decision Node - Divided by threshold 0.5 on feature[:,2] ('Astigmatic')
|----> Leaf node (<= 0.5)
| |--> class: ['no lenses' 'soft'] with probs [0.2 0.8]
|----> Leaf node (> 0.5)
| |--> class: ['hard' 'no lenses'] with probs [0.6 0.4]
```

Se puede ver que el árbol es bastante interpretable. Se llega a la conclusión de que si el *tear rate* es reducido, entonces lo más seguro es que no hacen falta gafas. Por otro lado, con un *tear rate* normal, que se lleven gafas fuertes o suaves depende de si se es astigmático o no.

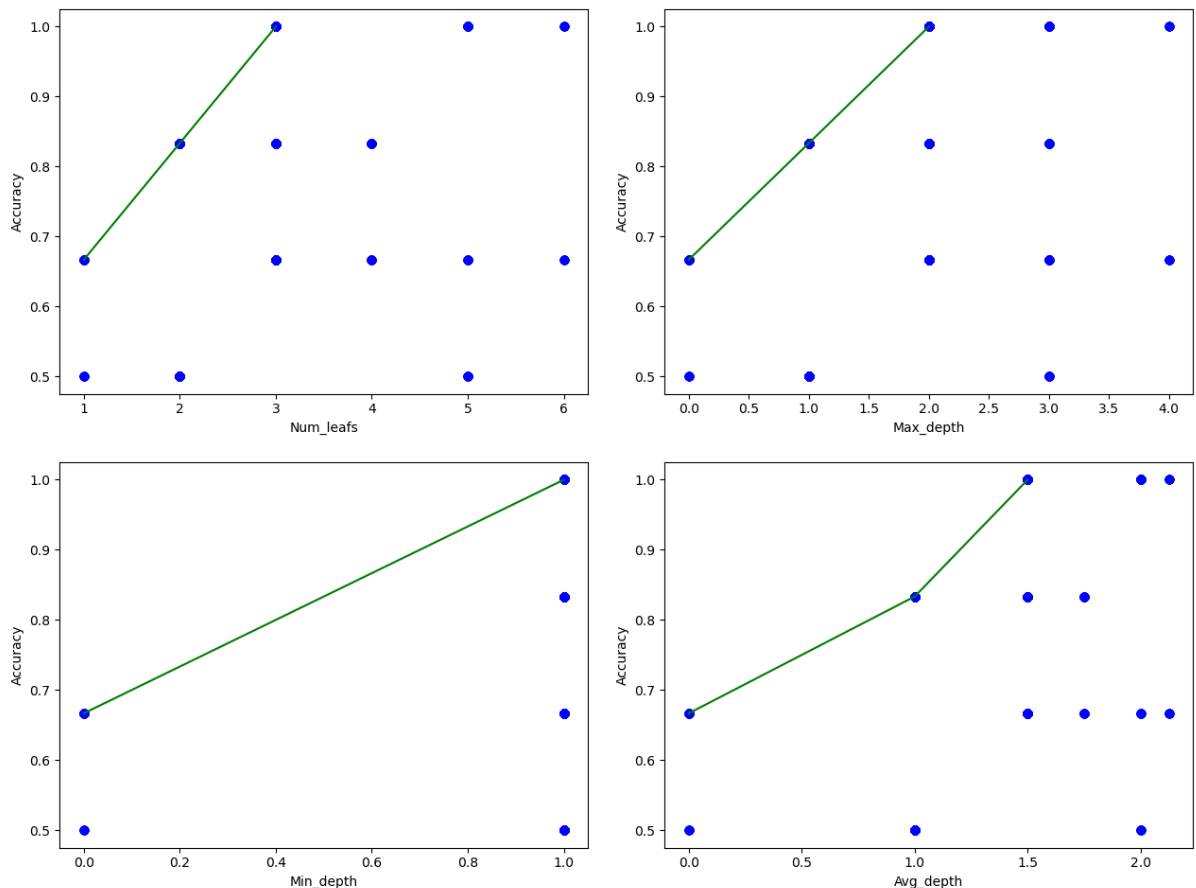


Figura 1: Frentes de pareto para distintos tipos de árboles entrenados con distintos hiperparámetros y distintos *folds* del dataset `lenses.csv`.

Estas mismas pruebas en el *dataset* `lenses2.csv` da resultados muy parecidos. De hecho, como la única diferencia es que se usa un valor numérico para la edad en vez de clases discretas, no hay diferencia para el árbol anterior de 3 hojas (pues dicho árbol no depende de la edad).

```
hiperparametros = {
    "min_info_gain": (0,0.05,0.1),
    "max_depth": (2,3,4,5,6,7,8,9,10),
    "min_samples_split": (2,3,4,5,6,7,8,9,10),
    "min_samples_leaf": (1,2,3,4,5,6,7,8,9,10),
}
results = CV(X, Y, hiperparametros, splits=4, th=Threshold(type="custom", th_array=[
    [20.0, 30.0, 40.0, 50.0, 60.0], # Age [17-70]
    [0.5], # Prescription [0-1]
    [0.5], # Astigmatic [0-1]
    [0.5], # Tear Rate [0-1]
]))
```

Por ver más en detalle otro árbol que utilice la edad, se muestra el siguiente árbol de 5 hojas (que tiene también una precisión del 100% en el *cross validation*). Como se puede ver, es una extensión del árbol de 3 hojas que tiene en cuenta la edad en el caso de que no se sea astigmático, y las prescripción en el caso de que lo sea.

```

|> (root) Decision Node - Divided by threshold 0.5 on feature 3 ('TearRate')
|--> Leaf node (<= 0.5)
|   |--> class: ['no lenses'] with probs [1.]
|--> (> 0.5) Decision Node - Divided by threshold 0.5 on feature 2 ('Astigmatic')
|   |--> (<= 0.5) Decision Node - Divided by threshold 40.0 on feature 0 ('Age')
|   |   |--> Leaf node (<= 40.0)
|   |   |   |--> class: ['soft'] with probs [1.]
|   |   |--> Leaf node (> 40.0)
|   |   |   |--> class: ['no lenses' 'soft'] with probs [0.5 0.5]
|   |--> (> 0.5) Decision Node - Divided by threshold 0.5 on feature 1 ('Prescription')
|   |   |--> Leaf node (<= 0.5)
|   |   |   |--> class: ['hard'] with probs [1.]
|   |   |--> Leaf node (> 0.5)
|   |       |--> class: ['hard' 'no lenses'] with probs [0.33333333 0.66666667]

```

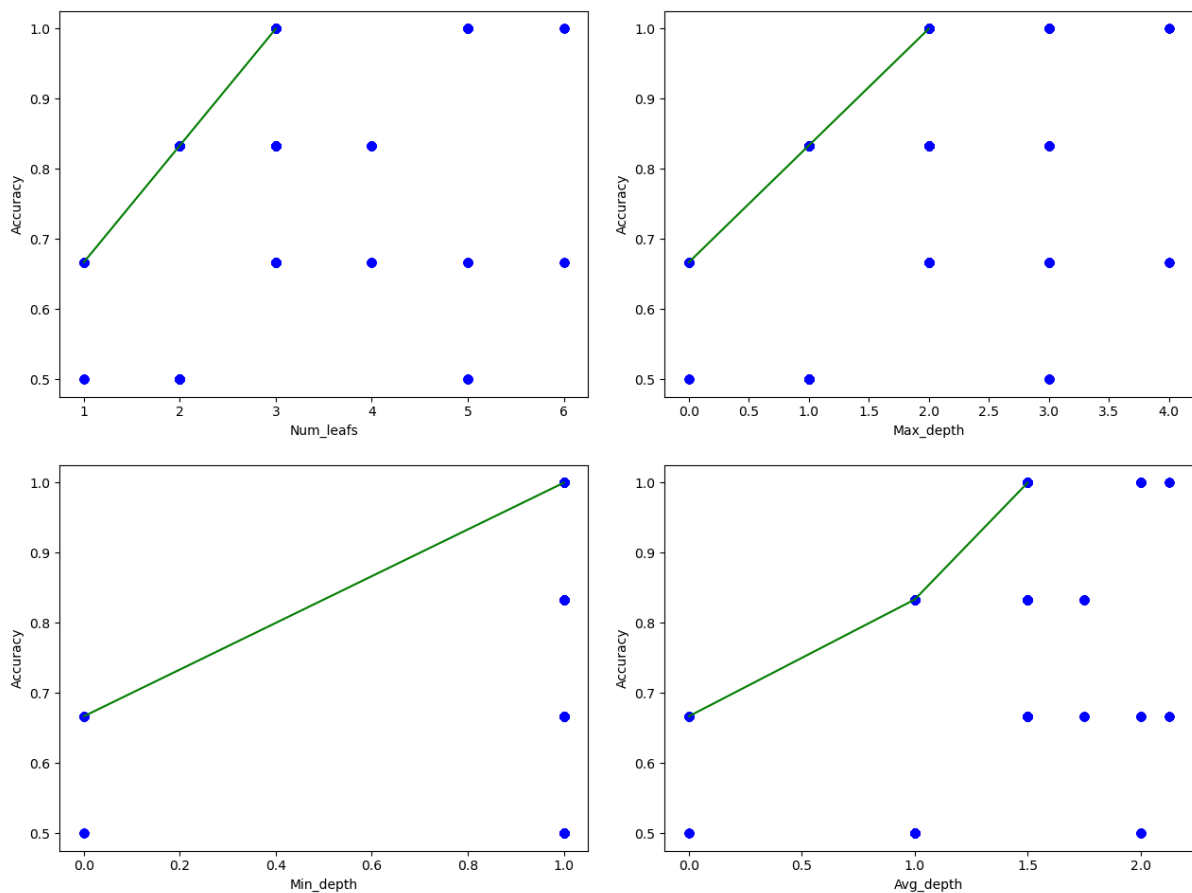


Figura 2: Frentes de pareto para distintos tipos de árboles entrenados con distintos hiperparámetros y distintos *folds* del dataset `lenses2.csv`.

En conclusión, hay varios árboles que tienen un buen balance entre precisión e interpretabilidad. Los dos vistos anteriormente tienen una precisión elevada, pero son muy simples y fáciles de entender.

4.e) Comparación con un MLP

A mayores vamos a realizar una comparación del árbol de decisión con un MLP (*Multi-Layer Perceptron*) entrenado sobre el mismo *dataset* `lenses.csv`. Para ello realizaremos un *cross validation* de la misma forma que se hizo para los árboles de decisión.

```

warnings.filterwarnings("ignore")
@ignore_warnings(category=ConvergenceWarning)
def mlp_CV(X, Y, hiperparametros : dict, splits=8):
    fuera_cv = StratifiedKFold(n_splits=splits, shuffle=True, random_state=0)

    hp_combinaciones = list(dict_cartesian_product(hiperparametros))
    resultados = []

    sca = StandardScaler()
    X = sca.fit_transform(X, Y)
    X = pd.DataFrame(X)

    for ixs_train_fuera, ixs_test_fuera in fuera_cv.split(X, Y):
        # Datos de entrenamiento para el bucle exterior
        X_train_fuera = X.iloc[ixs_train_fuera]
        Y_train_fuera = Y.iloc[ixs_train_fuera]

        # Datos de test del bucle exterior
        X_test_fuera = X.iloc[ixs_test_fuera]
        Y_test_fuera = Y.iloc[ixs_test_fuera]

        for i, params in enumerate(hp_combinaciones):
            try:
                model = MLPClassifier(**params)
            except:
                # Si la combinación de parámetros no
                # es correcta, continuar
                continue

            model.fit(X_train_fuera, Y_train_fuera)

            y_pred = model.predict(X_test_fuera)
            acc = metrics.accuracy_score(
                y_true=Y_test_fuera.to_numpy(),
                y_pred=y_pred
            )

            resultados.append({
                **params,
                'accuracy': acc,
                'model': model,
            })

    return resultados

```

Se probarán todas las combinaciones de parámetros de la lista que se muestra a continuación.

```

hiperparametros_mlp = {
    "hidden_layer_sizes": ([4], [8], [3,3], [4,4], [4,8,4], [3,6,3]),
    "max_iter": (200, 400, 600, 800, 1000),
    "learning_rate_init": (0.001, 0.01, 0.1),
    'solver': ('adam', 'sgd'), # Optimization algorithms
}

```

```
r_mlp = mlp_CV(X, Y, hiperparametros_mlp)
r_mlp = pd.DataFrame(r_mlp)
```

Los resultados conseguidos son los siguientes (se muestran sólo los entrenamientos que tuviesen las precisiones más altas):

```
percentil_90 = r_mlp['accuracy'].quantile(0.9)
sub_df = r_mlp[r_mlp['accuracy'] >= percentil_90]
sub_df
```

✓ 0.0s

	hidden_layer_sizes	max_iter	learning_rate_init	solver	accuracy	model
2	[4]	200	0.010	adam	1.0	MLPClassifier(hidden_layer_sizes=[4], learning...
3	[4]	200	0.010	sgd	1.0	MLPClassifier(hidden_layer_sizes=[4], learning...
4	[4]	200	0.100	adam	1.0	MLPClassifier(hidden_layer_sizes=[4], learning...
5	[4]	200	0.100	sgd	1.0	MLPClassifier(hidden_layer_sizes=[4], learning...
7	[4]	400	0.001	sgd	1.0	MLPClassifier(hidden_layer_sizes=[4], max_iter...
...	...	...	...	...	...	...
1420	[3, 6, 3]	400	0.100	adam	1.0	MLPClassifier(hidden_layer_sizes=[3, 6, 3], le...
1432	[3, 6, 3]	800	0.100	adam	1.0	MLPClassifier(hidden_layer_sizes=[3, 6, 3], le...
1434	[3, 6, 3]	1000	0.001	adam	1.0	MLPClassifier(hidden_layer_sizes=[3, 6, 3], ma...
1437	[3, 6, 3]	1000	0.010	sgd	1.0	MLPClassifier(hidden_layer_sizes=[3, 6, 3], le...
1438	[3, 6, 3]	1000	0.100	adam	1.0	MLPClassifier(hidden_layer_sizes=[3, 6, 3], le...

468 rows x 6 columns

Se puede ver que hay muchas combinaciones de parámetros que consiguen un 100% de precisión en el test. Sin embargo, al ser el test tan pequeño, poco podemos deducir de esta información. Aunque seguramente los modelos deberían ir bien.

La Figura 3 muestra la comparación de resultados para todos los modelos entrenados, para todas las combinaciones de hiperparámetros y *folds* del *dataset*.

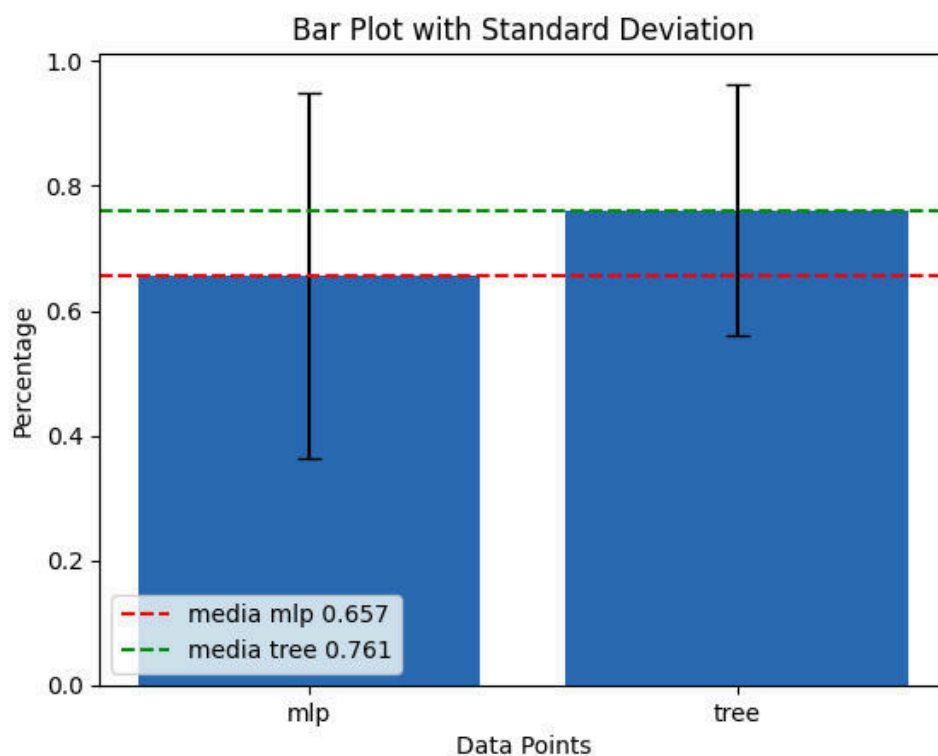


Figura 3: Comparación del MLP con el árbol de decisión para el *dataset* lentes.csv

De esta imagen podemos deducir que para un *dataset* de tamaño tan pequeño, el árbol construirá modelos más claros e interpretables, con mejores resultados en promedio, que el MLP.

Haciendo un CV probando una cantidad de combinaciones de parámetros inferior, acotando en base a los datos obtenidos de las pruebas anteriores tanto para el MLP y nuestro árbol pero con el iris podemos comprobar los siguiente:

```
iris_data = pd.read_csv("Iris.csv").drop("Id", axis=1)
iris_data["Species"] = iris_data["Species"].map({'Iris-setosa': 0, "Iris-versicolor": 1, "Iris-virginica": 2})
X = pd.DataFrame(iris_data.iloc[:, :-1].values)
Y = pd.DataFrame(iris_data.iloc[:, -1].values.reshape(-1,1))

hiperparametros = {
    "min_info_gain": (0,),
    "max_depth": (3,4,5,6,7),
    "min_samples_split": (2, 4, 6, 8),
    "min_samples_leaf": (1,2,3,4),
}

results = CV(X, Y, hiperparametros, splits=4)
results = pd.DataFrame(results)

hiperparametros_mlp = {
    "hidden_layer_sizes": ([4], [8], [3,3], [3,6,3]),
    "max_iter": (200, 600, 1000),
    "learning_rate_init": (0.01, 0.1),
    'solver': ('adam',), # Optimization algorithms
}

r_mlp = mlp_CV(X, Y, hiperparametros_mlp)
r_mlp = pd.DataFrame(r_mlp)
```

En la Figura 4 se muestran los resultados de evaluar la precisión de todos los árboles y MLPs construidos para la prueba de Iris, probando todas las combinaciones de hiperparámetros.

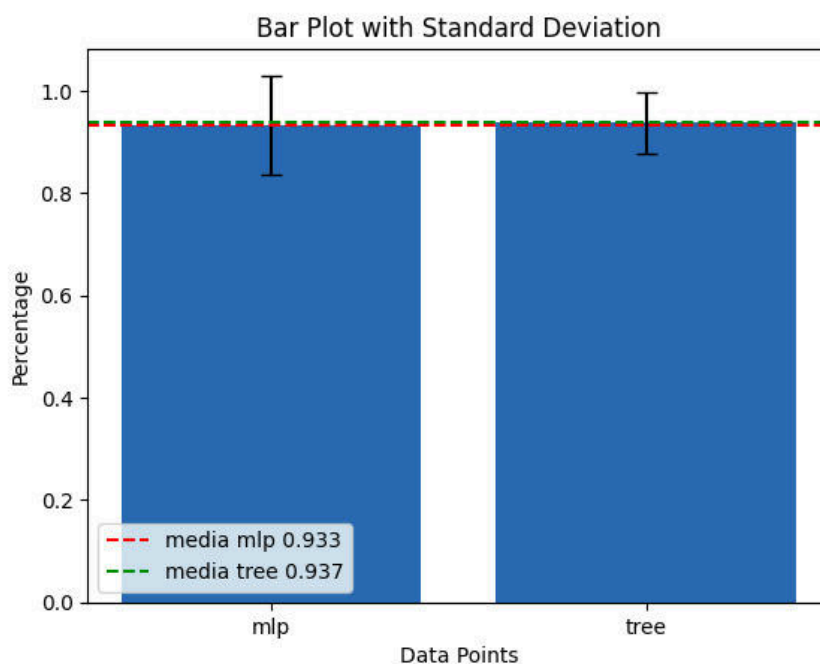


Figura 4: Comparación del MLP con el árbol de decisión para el *dataset* Iris.csv

Vemos que el árbol tiene aproximadamente la misma media y menos desviación típica, lo cual nos puede dar a entender que para este problema en concreto el árbol es ligeramente mejor, pero no por un largo margen. Sin embargo, empíricamente podemos ver que a veces el MLP es mejor.

De esta manera podemos concluir que para este problema la principal razón por la que se usaría un árbol es para poder entender las decisiones de los razonamientos, pues, como se puede ver en el siguiente árbol de 5 hojas y de precisión del 100%, los resultados son bastante interpretables (se clasifica en distintos tramos de longitud, y para el tramo de longitud entre 4.75cm y 5.15cm, se clasifica por ancho).

```
|> (root) Decision Node - Divided by threshold 2.45 on feature 2 ('PetalLengthCm')
|--> Leaf node (<= 2.45)
|   |--> class: [0.] with probs [1.]
|--> (> 2.45) Decision Node - Divided by threshold 4.75 on feature 2 ('PetalLengthCm')
|----> Leaf node (<= 4.75)
|       |--> class: [1. 2.] with probs [0.97142857 0.02857143]
|----> (> 4.75) Decision Node Divided by threshold 5.15 on feature 2 ('PetalLengthCm')
|-----> (<= 5.15) Decision Node Divided by threshold 1.75 on feature 3 ('PetalWidth')
|-----> Leaf node (<= 1.75)
|           |--> class: [1. 2.] with probs [0.6 0.4]
|-----> Leaf node (> 1.75)
|           |--> class: [1. 2.] with probs [0.1 0.9]
|-----> Leaf node (> 5.15)
|           |--> class: [2.] with probs [1.]
```

5) Conclusión

La primera conclusión que podemos sacar del trabajo es la dificultad de trabajar con *datasets* pequeños. Por un lado, dificultan mucho el entrenamiento de modelos como el MLP, por otro lado, tener *datasets* pequeños limita la amplitud de testeo que se puede hacer, limitando las conclusiones que se pueden sacar.

La segunda conclusión que podemos sacar es que para conjuntos de datos pequeños, modelos de IA más simples que un MLP, como un árbol de decisión en este caso, producen resultados relativamente mejores y más estables. Además, en el caso del árbol de decisión en particular, los modelos son fácilmente interpretables, de manera que son mucho más sencillos de entender, y sacar conclusiones, no solo de los resultados, sino también de la construcción. Esto permite, por ejemplo, comprobar si la lógica aprendida por el modelo es realmente correcta, ya que en el caso de que no lo sea, difícilmente funcionará correctamente el modelo en casos reales.

En relación a la primera y la segunda conclusión, podemos probar que no todos los modelos sirven para todos los problemas, ya que el carácter y los datos que tengamos va a influir en los resultados. Por ejemplo, por mucho que un clasificador de tipo ROBERTA sea mejor que una regresión logística, para entrenar a un ROBERTA necesitas miles de datos, por lo que si no los tienes, entonces una regresión logística puede que sea mejor.